

FORENIX-OSS | V0.1 | 2026-05

forenix-oss

How-To Guide

Author: Ali Murtaza Bhutto | **Repository:** github.com/thunderstornX/forenix-oss | **License:** MIT

forenix-oss - How-To Guide

Cookbook-style recipes. Each topic is independent. Skim the index, jump to the one you need.

Index

Investigations

- [How to create a new investigation](#)
- [How to open an existing investigation](#)
- [How to filter the investigations list](#)
- [How to delete or archive an investigation](#)

The Pipeline

- [How to run the OSINT pipeline against a target](#)
- [How to pick which agent groups to run](#)
- [How to run the pipeline against a real LLM \(NVIDIA or OpenRouter\)](#)
- [How to compare two AI adapters on the same target](#)

Findings

- [How to verify a finding](#)
- [How to promote a finding to forensic evidence](#)
- [How to view a finding's reasoning trace](#)

Forensic cases

- [How to bridge an investigation to a case](#)
- [How to create a case from scratch](#)
- [How to open an existing case](#)
- [How to filter the cases list](#)

Evidence + chain of custody

- [How to seal an evidence row](#)
- [How to read an evidence commit chain](#)
- [How to read the branch graph](#)

Audit + integrity

- [How to verify the audit chain](#)
- [How to read the audit log](#)
- [How to verify the chain offline \(12 lines of Python\)](#)

Reports

- [How to read a report](#)
- [How to generate a new report](#)

Graphs

- [How to read the entity graph](#)
- [How to read the cross-case network graph](#)

Navigation + UX

- [How to use the command palette \(⌘K\)](#)
- [How to switch views with keyboard shortcuts](#)
- [How to collapse the sidebar](#)
- [How to deep-link to a view or detail panel](#)

Setup + ops

- [How to switch the active AI adapter](#)
- [How to re-seed the database](#)
- [How to recover from a 'pipeline run hung' state](#)

Investigations

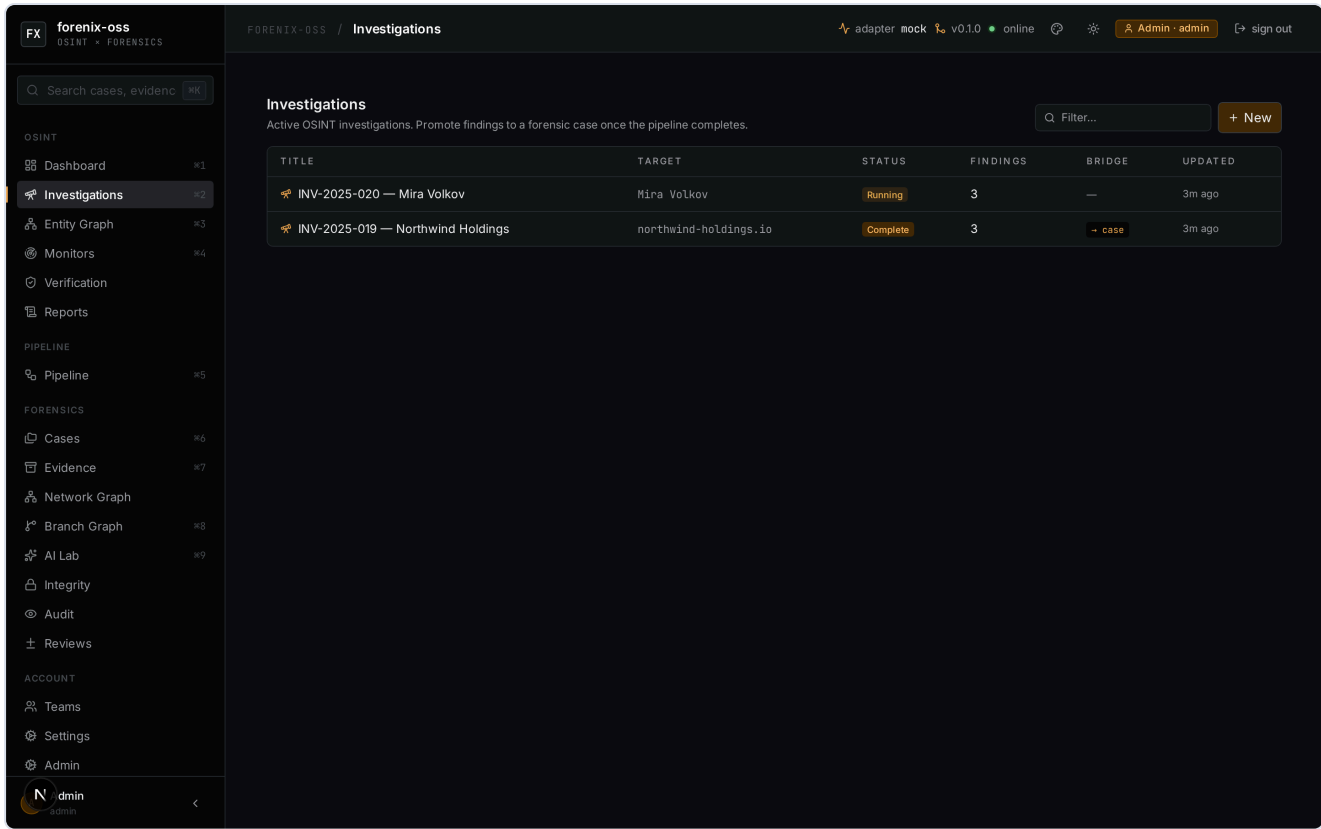
How to create a new investigation

Goal. Open a fresh OSINT workspace bound to a target.

Steps.

1. Click **Investigations** in the sidebar (or press **⌘2**).
2. Click the **+ New** button in the top-right of the table.
3. Fill in:
 - **Title** - a short identifier (e.g. `INV-2026-008 - Acme Holdings`).
 - **Target** - the actual thing you're investigating (`acme-holdings.io`).
 - **Target type** - pick one of `domain` , `person` , `organization` , `ip` , `username` , `phone` .
 - **Objective** - one or two sentences on what you're trying to find out.
4. Click **Create**.

Result. A new investigation lands at the top of the list with status `draft` . An audit row (`create_investigation`) is written to the chain.

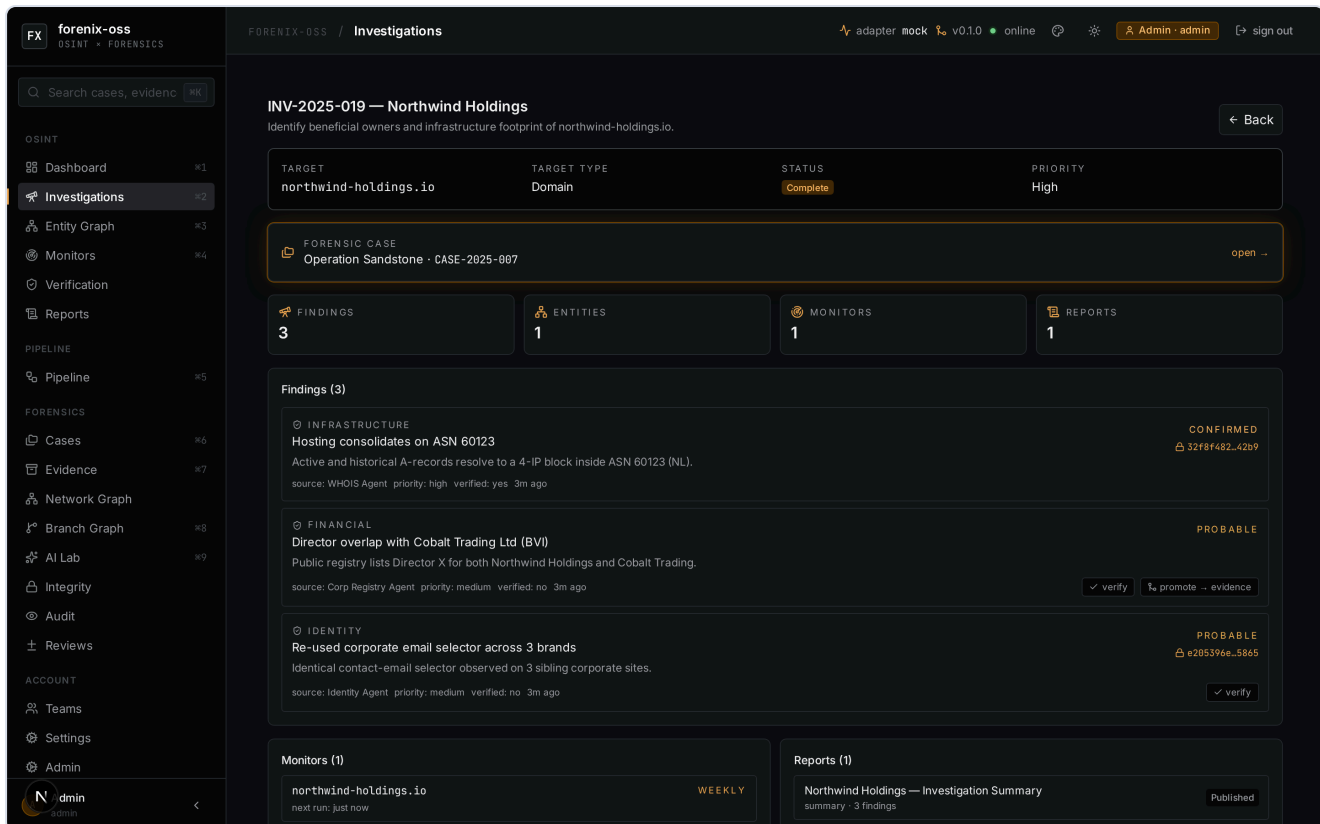


How to open an existing investigation

Steps.

1. Open the Investigations view (⌘2).
2. Click any row in the table.

The detail panel slides in. Use <- **Back** in the top-right to return to the list.



How to filter the investigations list

Steps.

1. Click **Investigations** (⌘2).
2. Click the **Filter...** input above the table.
3. Type any of: title fragment, target hostname/IP, target type, status (`draft` , `running` , `complete` , `paused` , `archived`), or priority (`low` , `medium` , `high` , `critical`).

The table narrows in real time. Clear the filter to see everything again.

How to delete or archive an investigation

Not yet exposed as a UI button in v0.1. For now:

- Open a Bun REPL or use Prisma Studio (`bunx prisma studio`).
- Set `status = "archived"` to hide it from primary lists.
- A future release will expose this directly on the detail panel.

The Pipeline

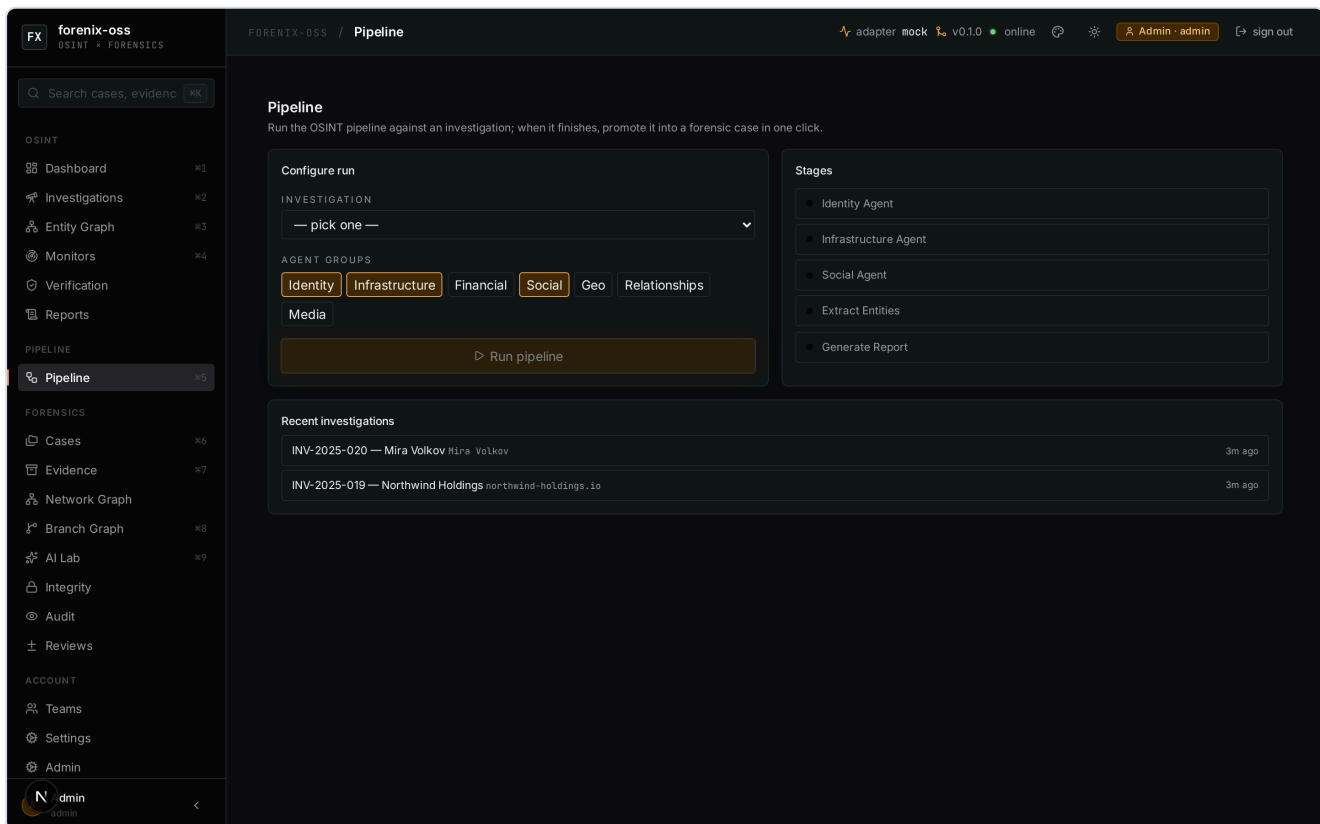
How to run the OSINT pipeline against a target

Goal. Have the AI adapter analyse the target across several agent groups, extract entities, and draft a report - all in one button.

Steps.

1. Click **Pipeline** (§5).
2. Pick an investigation from the dropdown.
(Or click any row in **Recent investigations** below - it sets the selection too.)
3. Toggle the **agent groups**. The chips at top: `identity` , `infrastructure` , `financial` , `social` , `geo` , `relationships` , `media` . Three or four is the usual sweet spot.
4. Click **Run pipeline**.

Result. Right-hand panel animates each stage from `idle` -> `running` -> `done` . The pipeline writes findings, extracts entities, and generates a report. Counts appear in the "Pipeline complete" card.



How to pick which agent groups to run

Group	Best for
identity	unique people / aliases / PII clusters
infrastructure	domains, ASN/IP footprints, registrar lineage
financial	shell companies, sanctions adjacency, crypto
social	forum personas, follower cliques, mention networks
geo	EXIF clusters, location pins, satellite cross-refs
relationships	co-travelers, email/thread peers, office mates
media	image / video / perceptual-hash matches

Cost note. Each group is one separate LLM call. Three groups is ~ 3x the latency of one. Use only what's relevant to the target type.

How to run the pipeline against a real LLM (NVIDIA or OpenRouter)

One-shot, per-request.

Open a terminal and POST against the API with an `adapter` field:

```
INV=$(curl -s http://localhost:3000/api/investigations | jq -r '.data[0].id')
curl -X POST -H "content-type: application/json" \
  -d '{"agentGroups":["identity","social","geo"],"adapter":"nvidia"}' \
  http://localhost:3000/api/pipeline/run/$INV
```

The response shape:

```
{
  "data": {
    "investigationId": "...",
    "adapter": "nvidia",
    "agentGroups": ["identity","social","geo"],
    "findings": 11,
    "entities": 5,
    "relations": 7,
    "report": { "id": "...", "title": "..." }
  }
}
```

Permanent - for every request.

Edit `.env` :

```
AI_ADAPTER=nvidia
NVIDIA_API_KEY=nvapi-...
NVIDIA_MODEL=meta/llama-3.3-70b-instruct
```

...and restart the dev server (`Ctrl+C` then `bun run dev`).

The top-bar adapter chip will flip from `mock` to `nvidia` .

How to compare two AI adapters on the same target

Goal. A/B two models and see which one produces better findings on a specific target.

Steps.

1. Pick an investigation. Note its id.
2. Run the pipeline through adapter A:

```
curl -X POST -H "content-type: application/json" \
  -d '{"agentGroups":["identity","social"],"adapter":"nvidia"}' \
  http://localhost:3000/api/pipeline/run/$INV
```

3. Open the investigation detail (⌘2 -> click). Note the findings.
4. Re-run with adapter B:

```
curl -X POST -H "content-type: application/json" \
  -d '{"agentGroups":["identity","social"],"adapter":"openrouter"}' \
  http://localhost:3000/api/pipeline/run/$INV
```

5. Refresh the detail. The second batch is appended; the *sourceName* field on each finding tells you which adapter produced it.

Findings

How to verify a finding

Steps.

1. Open the parent investigation (⌘2 -> click).
2. Scroll to the **Findings** section.
3. On any unverified row, click **verify** on the right.

Result. A toast confirms; the finding's `verified` flag flips to `true` ; the confidence is upgraded to `confirmed` if it was `unverified` . An audit row (`verify_finding`) is written.

How to promote a finding to forensic evidence

Prerequisite. The investigation must already be bridged to a case. If it isn't, the **promote -> evidence** button does not show (use [How to bridge an investigation to a case](#) first).

Steps.

1. Open the investigation detail.
2. On any finding that hasn't been promoted, click **promote -> evidence**.

Result.

- A new Evidence row appears on the linked case, with SHA-256 hash and an initial `add:` commit on `main`.
- The finding's `evidenceId` is stamped - the row shows the green hash chip next time you open it.
- An audit row (`promote_finding_to_evidence`) is written.

If you don't see the button on a finding row, it means one of:

- the investigation isn't bridged yet, **or**
- the finding has already been promoted (look for the green hash chip on the right of the row).

How to view a finding's reasoning trace

The reasoning trace is the agent's free-text explanation of why it surfaced the finding.

Steps.

1. Open the investigation detail.
2. Scroll to **Findings**.
3. Each finding row shows the trace inline under the description.

In the database, the trace lives in `Finding.reasoningTrace` . It is included in every report the platform generates.

Forensic cases

How to bridge an investigation to a case

Goal. Turn an OSINT investigation into a forensic case that holds the findings as Evidence.

Two paths - pick one.

Path A: from the Pipeline view (recommended).

1. Run the pipeline (see [above](#)).
2. When the *Pipeline complete* card appears, click **Open forensic case ->**.

Path B: by API call.

```
curl -X POST -H "content-type: application/json" \
  -d '{"promoteFindings":true}' \
  http://localhost:3000/api/bridge/inv-to-case/$INV
```

Result. Either path:

- mints a Case with an auto `CASE-YYYY-NNN` number,
- creates a `main` branch,
- promotes every finding to Evidence (because `promoteFindings: true`),
- sets `Investigation.caseId` (the green chip appears on the investigation detail),
- writes `bridge_investigation_to_case` + `findings_promoted_to_evidence` audit rows.

If you want the case but **don't** want findings auto-promoted, pass `"promoteFindings": false` in the API body. The button on the Pipeline view always promotes.

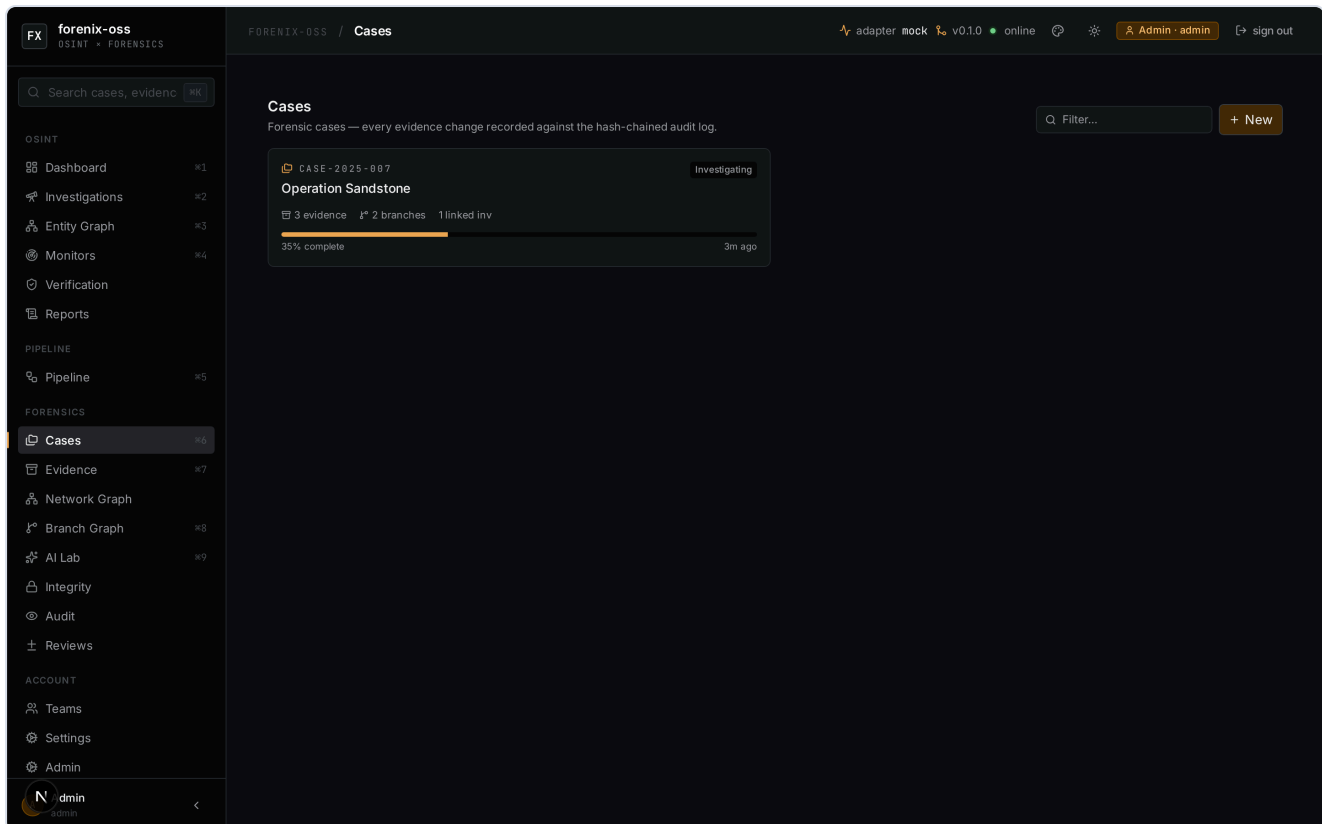
How to create a case from scratch

When you want a forensic case that isn't tied to an investigation (e.g. evidence already in hand).

Steps.

1. Click **Cases** (⌘6).
2. Click **+ New**.
3. Fill in:
 - **Title** - `Operation Foo`.
 - **Description** - what this case is about.
 - **Priority** - `low` / `medium` / `high` / `critical`.
4. Click **Create**.

Result. A case lands at the top of the grid with status `open`, an auto `CASE-YYYY-NNN` number, and a fresh `main` branch.



How to open an existing case

1. Click **Cases** (⌘6).
2. Click any card.

The detail panel slides in.

The screenshot shows the 'forenix-oss' web application interface. The top navigation bar includes the 'forenix-oss' logo, a search bar, and user information. The sidebar on the left lists various sections: OSINT (Dashboard, Investigations, Entity Graph, Monitors, Verification, Reports), PIPELINE (Pipeline), and FORENSICS (Cases, Evidence, Network Graph, Branch Graph, AI Lab, Integrity, Audit, Reviews). The 'Cases' section is currently selected.

The main content area for 'Operation Sandstone' shows a cross-jurisdictional shell-corporation network linked to a series of OSINT leads from Investigation INV-2025-019. A table displays case details:

CASE #	STATUS	PRIORITY	PROGRESS
CASE-2025-007	Investigating	High	35%

Below the table, there are sections for 'Linked investigations (1)', 'EVIDENCE' (3 items), 'BRANCHES' (2), 'MERGE REQUESTS' (0), and 'ASSIGNEES' (2). The 'Evidence (3)' section shows two items:

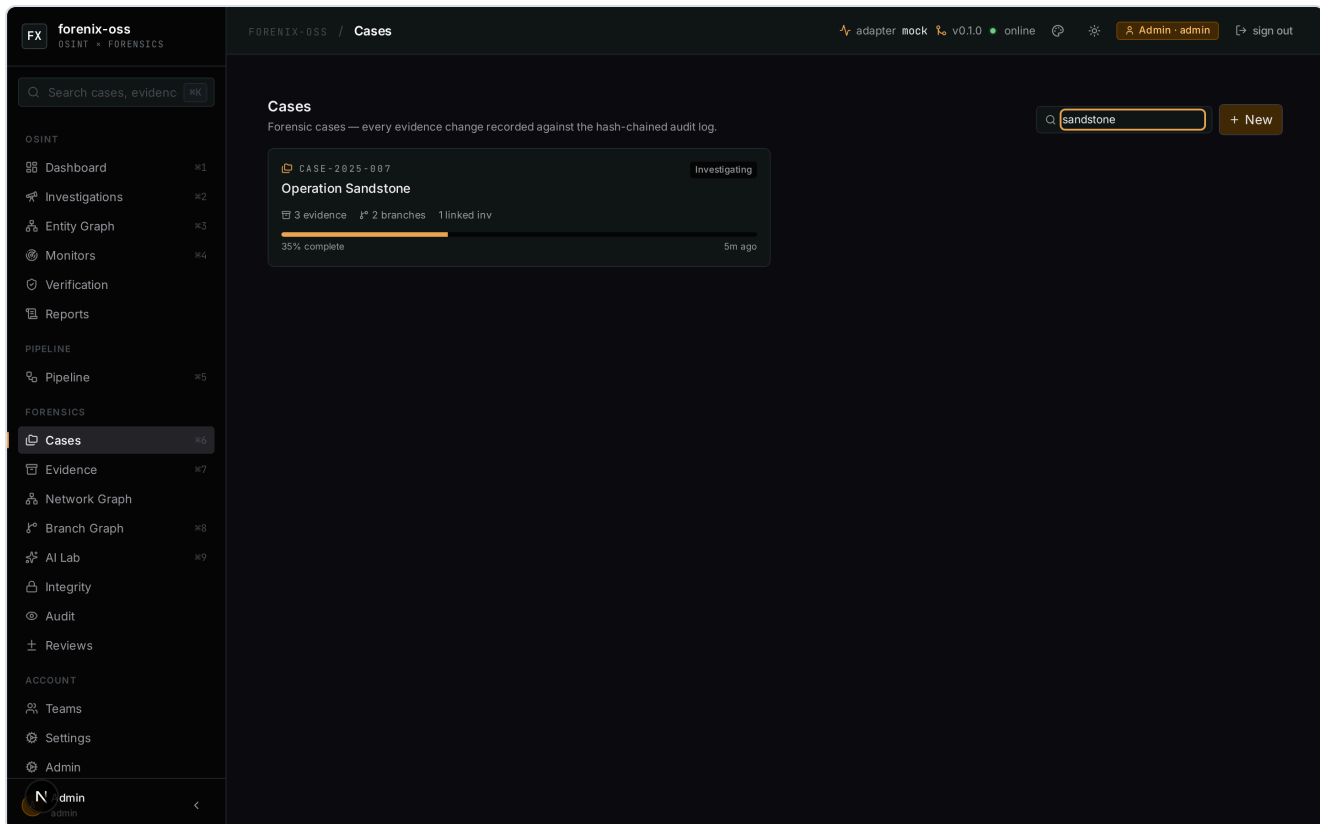
- pcap-handshake-fragment.pcap**: TLS ClientHello fragment captured during outbound call to suspect VPS. It includes a commit chain with two entries: '388c25d1.8767 ADD: pcap-handshake-fragment.pcap' and 'd3fa7230.4e8a VERIFY: verify: chain of custody validated for pcap-handshake-fragment.pcap'.
- whois-snapshot-northwind.json**: WHOIS pivot for northwind-holdings.io with contact-email reuse. It includes a commit chain with two entries: '07d63c2b.3ecb ADD: whois-snapshot-northwind.json' and '635bc6e1.0420 VERIFY: verify: chain of custody validated for whois-snapshot-northwind.json'.

How to filter the cases list

Same pattern as Investigations:

1. Click **Cases** (⌘6).
2. Type in the **Filter...** input above the grid.

Matches across title, case number, status, priority.



Evidence + chain of custody

How to seal an evidence row

A sealed row is **immutable** - no further commits are accepted. Seal when the case team has signed off and the evidence is ready for court.

Steps.

1. Open the parent case (⌘6 -> click).
2. On any evidence card, click **seal** on the right side of the header.

Result.

- The status chip flips from `collected` / `verified` to `sealed`.
- A `seal` commit is added to the case's `main` branch.
- An audit row (`seal_evidence`) is written.
- The **seal** button disappears (you can't seal twice).

How to read an evidence commit chain

Every evidence row carries its full commit chain inline on the case detail view.

Each row shows:

- a dot in the branch colour,

- the commit hash (truncated, monospace),
- the change type (`add` , `verify` , `seal` , `merge`),
- the commit message,
- the branch name in brackets,
- the relative time.

A green dot at the side means the commit has been verified by an analyst.

The screenshot shows the forenix-oss web application. The top navigation bar includes the logo, version (v0.1.0), status (online), and user information (Admin - admin). The sidebar on the left lists various features under OSINT, PIPELINE, and FORENSICS. The main content area is titled 'Cases' and shows details for 'Operation Sandstone'. It includes a case summary, linked investigations, evidence counts, and a detailed view of two evidence items with their commit chains and verification status.

How to read the branch graph

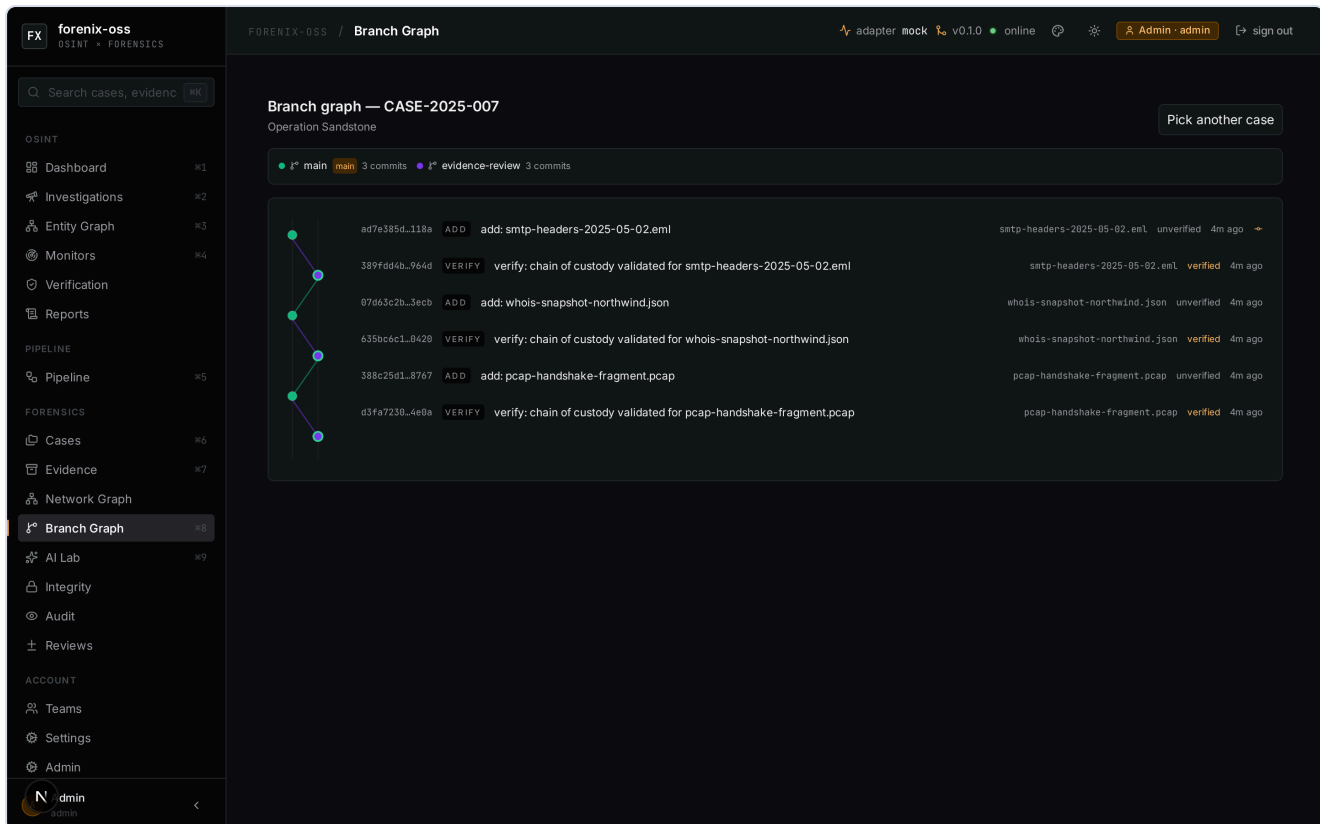
The Branch Graph is the visual replacement for "scroll through case-detail commits."

Steps.

1. Click **Branch Graph** (⌘8).
2. If no case is active, pick one from the list.

What you see.

- A **swimlane per branch** - leftmost is `main` .
- A **dot per commit**, branch-coloured.
- Verified commits get a **green ring**.
- The right panel shows the commit message, evidence name, change-type, verified state, and relative time.



To switch cases, click **Pick another case** in the top-right.

Audit + integrity

How to verify the audit chain

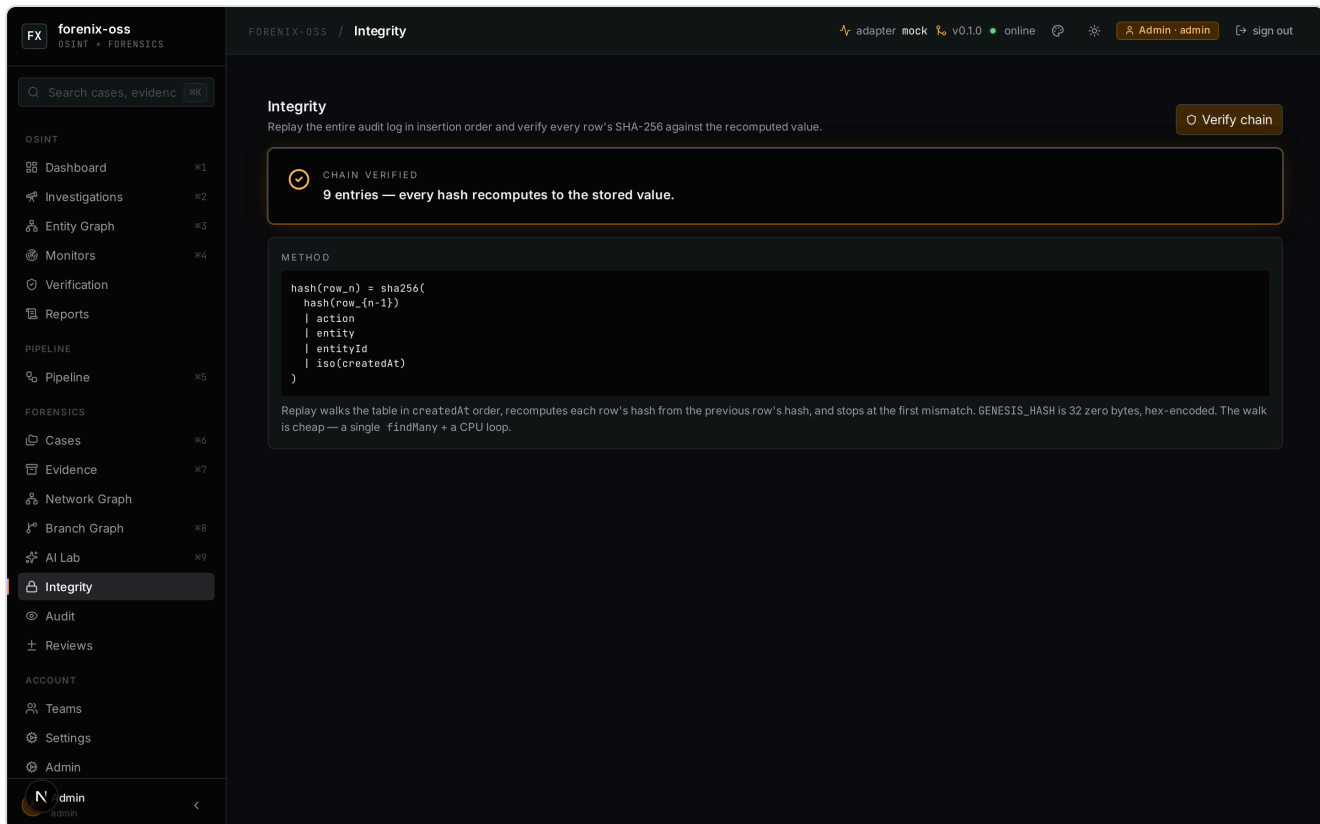
The single most important verification on the platform.

Steps.

1. Click **Integrity** in the sidebar.
2. Click **Verify chain**.

Result.

- Green panel: *"N entries - every hash recomputes to the stored value."* You're done. The chain is intact.
- Red panel: shows the first row id where the chain broke + expected vs. stored hashes side-by-side. Escalate per your incident-response policy.



How to read the audit log

Steps.

1. Click **Audit** in the sidebar.

Every row in chain order. Per row:

- **Hash** (truncated, monospace) - this row's hash.
- The previous row's hash inline below - the chain link.
- **Action** - `pipeline_started` , `promote_finding_to_evidence` , `seal_evidence` , etc.
- **Entity** - Investigation / Case / Finding / Evidence.
- **Scope** - clickable chips that jump to the related investigation and/or case.
- **When** - relative time.

Use the filter to narrow by action, entity, or hash fragment.

Audit log

Append-only chain across both workflows. Every write computes sha256(prevHash | action | entity | entityId | iso(t)).

Q: action, entity, hash... 9 entries

HASH	ACTION	ENTITY	SCOPE	WHEN
4b465bca..7f26 -- 80808080..0000	create_user	User cmp97rsz8000	—	5m ago
bb10de21..a104 -- 4b465bca..7f26	create_user	User cmp97rt01000	—	5m ago
1b117e08..6856 -- bb10de21..a104	create_user	User cmp97rt05000	—	4m ago
61830660..082e -- 1b117e08..6856	create_investigation	Investigation cmp97rt2c001	✳ inv	4m ago
52ad3c30..d86b -- 61830660..082e	run_pipeline	Investigation cmp97rt2c001	✳ inv	4m ago
04361111..7fc1 -- 52ad3c30..d86b	create_case	Case cmp97rt0f000	📁 case	4m ago
7003e873..d45b -- 04361111..7fc1	promote_to_evidence	Finding f-1	📁 case	4m ago
7c363f93..3a5d -- 7003e873..d45b	verify_evidence	EvidenceCommit cmp97rt0f000	📁 case	4m ago
bd73298b..44ab -- 7c363f93..3a5d	generate_report	Report r-1	✳ inv	4m ago

How to verify the chain offline (12 lines of Python)

Anyone with read access to the database can re-prove the chain without forenix-oss running.

```
import csv, hashlib
GENESIS = "0" * 64
prev = GENESIS
for r in csv.DictReader(open("audit.csv")):
    h = hashlib.sha256("|".join([
        prev, r["action"], r["entity"], r["entityId"] or "",
        r["createdAt"]
    ]).encode("utf-8")).hexdigest()
    assert r["prevHash"] == prev and r["hash"] == h, f"BROKEN at {r['id']}"
    prev = r["hash"]
print("chain OK")
```

Export `audit.csv` from Postgres (`COPY ("AuditLog") TO STDOUT CSV HEADER`) or SQLite (`.mode csv` + `SELECT * FROM AuditLog ORDER BY createdAt ASC`).

Reports

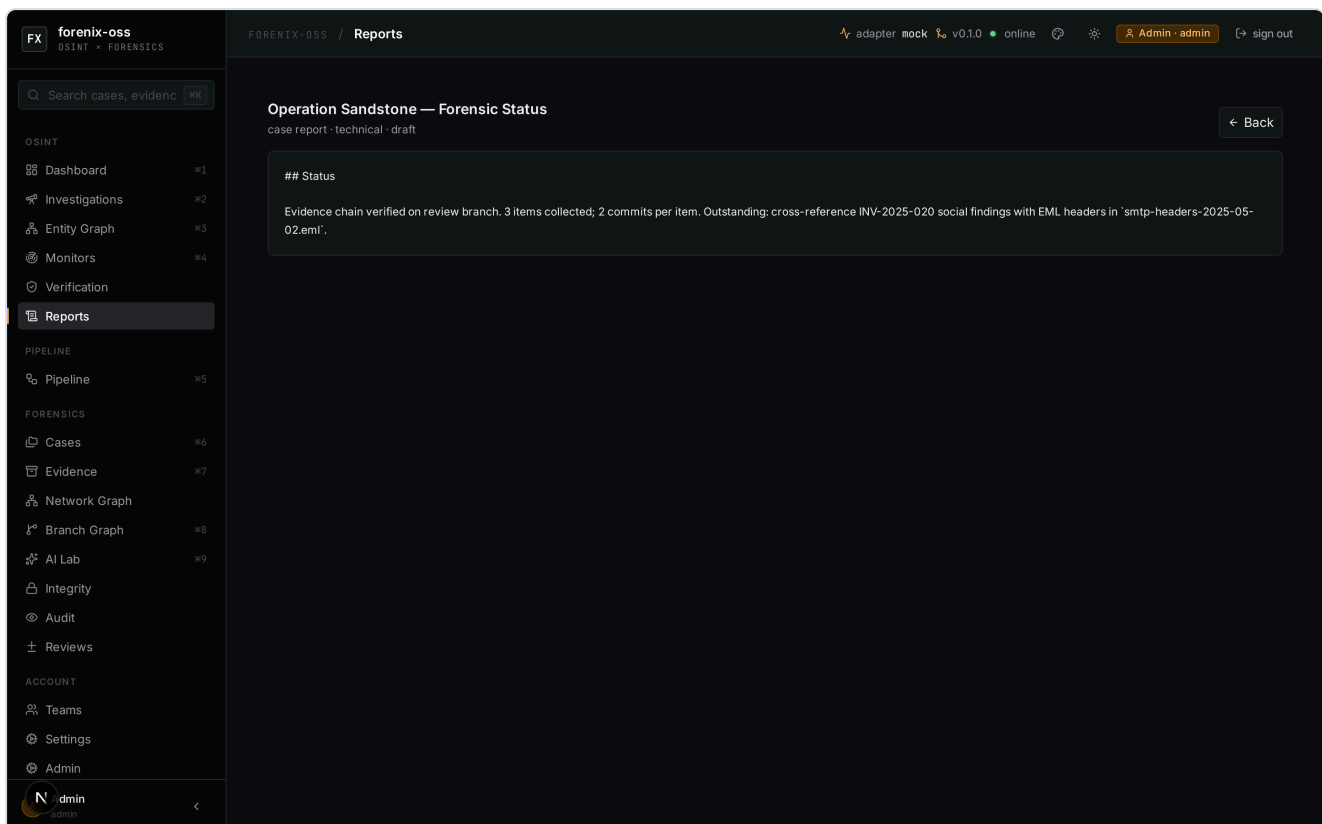
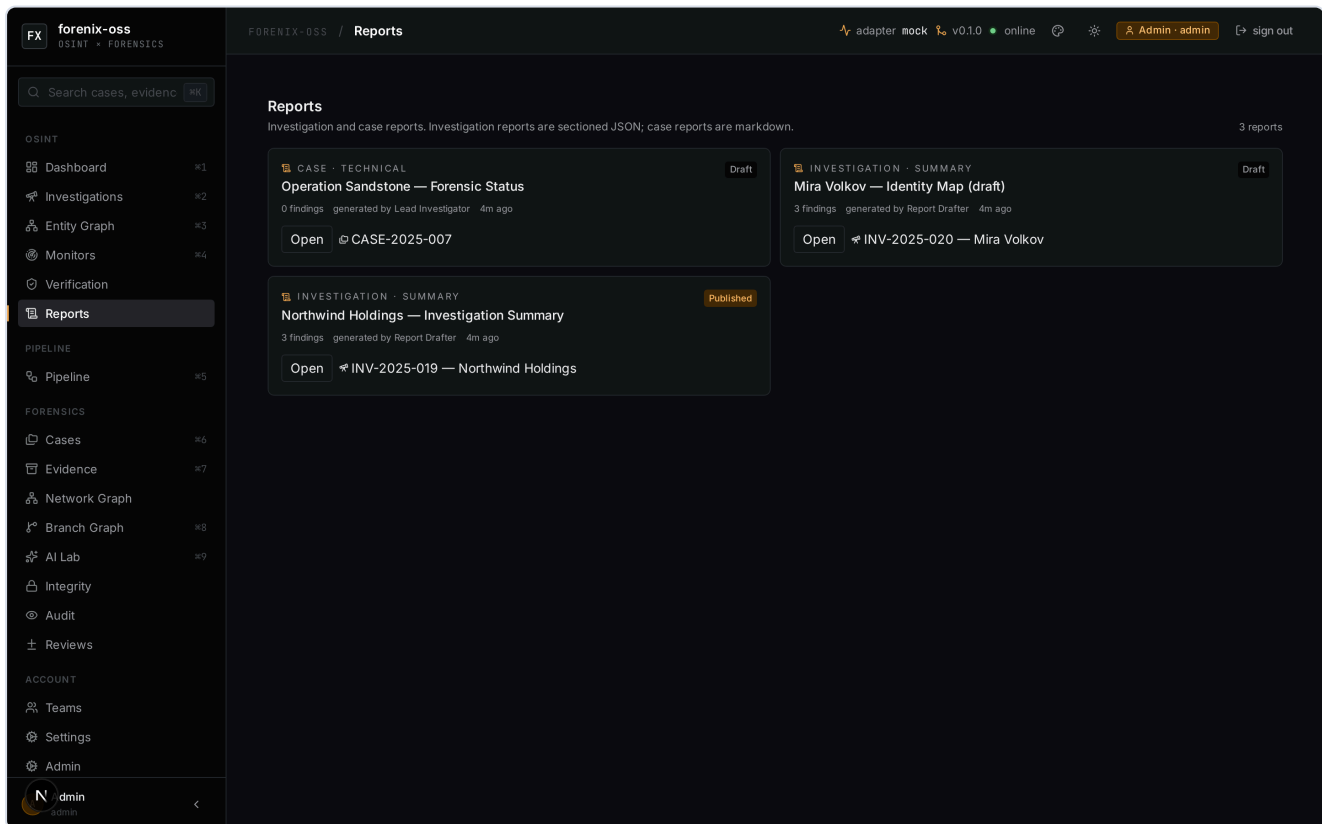
How to read a report

Steps.

1. Click **Reports** in the sidebar.

2. Click **Open** on any report card.

The detail view auto-detects whether the report has **sections** (JSON, Argus shape), **content** (markdown, ForenX shape), or both, and renders each accordingly.



Use **<- Back** in the top-right to return to the list.

How to generate a new report

Reports are produced **automatically** as the last stage of the Pipeline runner. You don't need to do anything special:

1. Run the pipeline (see above).
2. When it completes, the report is already saved with `source = "investigation"` and attached to the parent investigation.

To regenerate, just re-run the pipeline - a new report row is written each time.

Graphs

How to read the entity graph

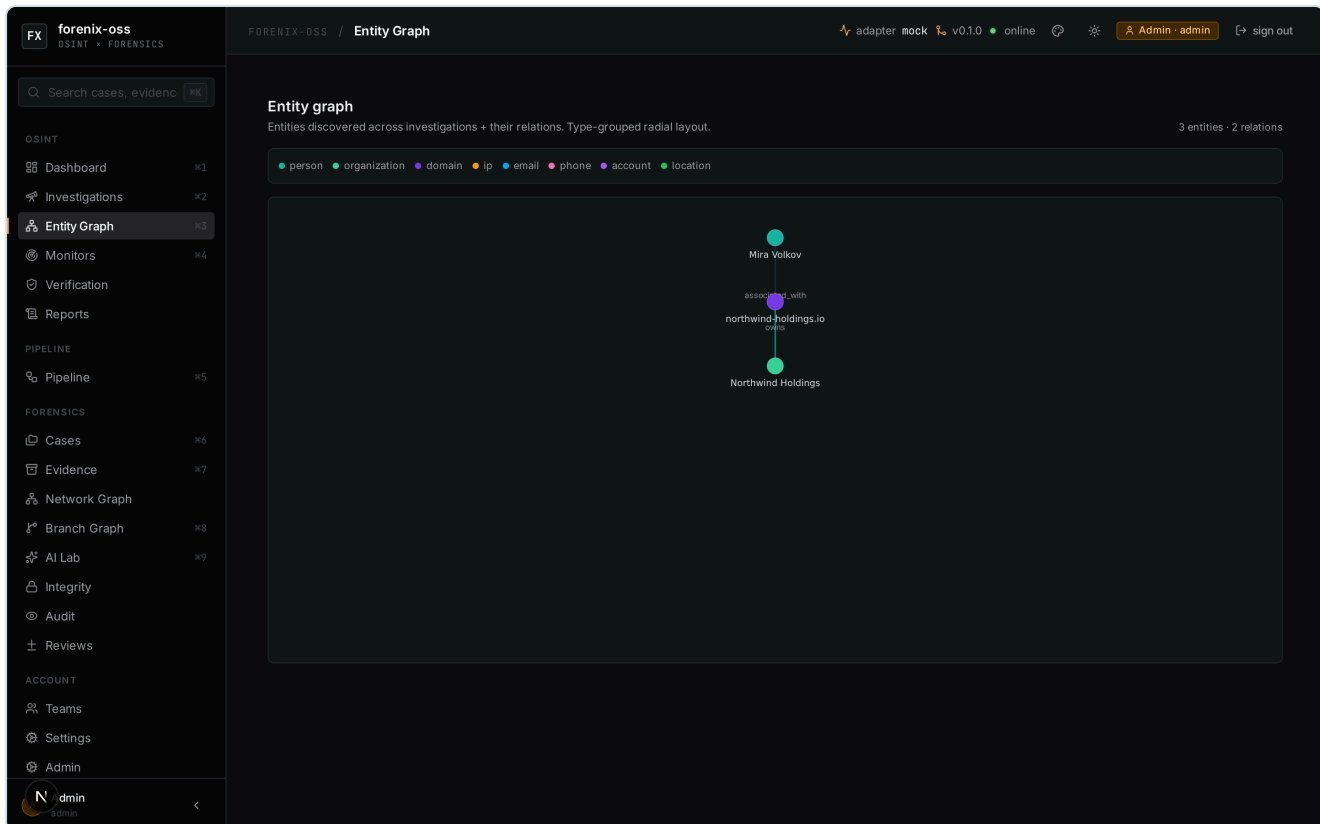
The entity graph shows OSINT entities + their relations, grouped by type on concentric rings.

Steps.

1. Click **Entity Graph** (⌘3).

Reading it.

- Each ring is one type (person / org / domain / ip / ...). Outer rings are the next type.
- A line between two entities is a relation. The line's brightness reflects the relation's `confidence` :
 - bright = `confirmed` ,
 - medium = `probable` ,
 - faint = `unverified` .
- The line label is the relation type (`owns` , `resolves_to` , `associated_with` , ...).



How to read the cross-case network graph

The Network Graph shows **everything** - users, agents, investigations, cases, evidence, entities - on one canvas.

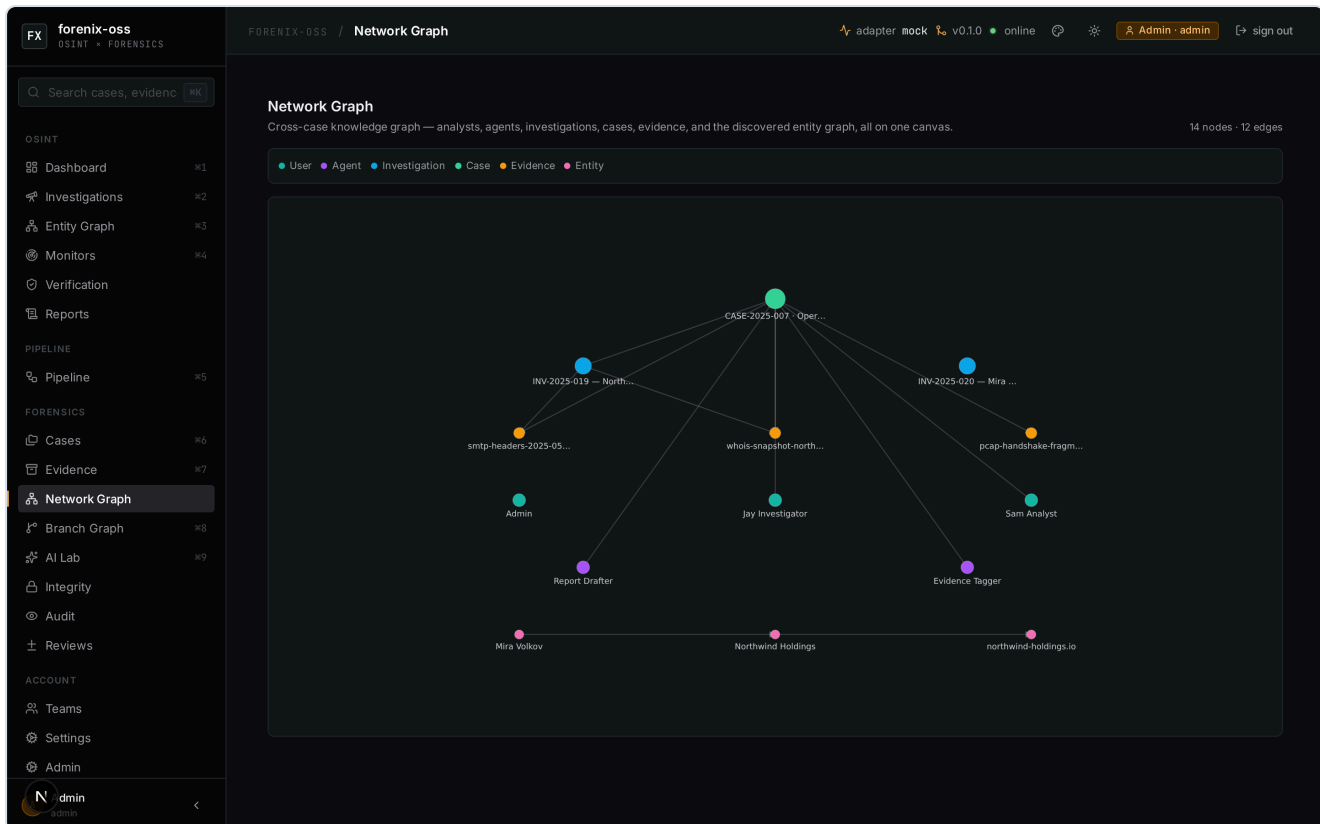
Steps.

1. Click **Network Graph** in the sidebar.

Reading it.

- Each row (swimlane) is one **kind** of node - case, investigation, evidence, user, agent, entity.
- Arrows go from cause to effect: `case -> evidence`, `investigation -> case` (the bridge), `user -> case` (assignment), `entity -> entity` (relation).
- Hover any node to see its full label in the tooltip.

This is the view to open when you want to see what *connects* two otherwise-unrelated cases.



Navigation + UX

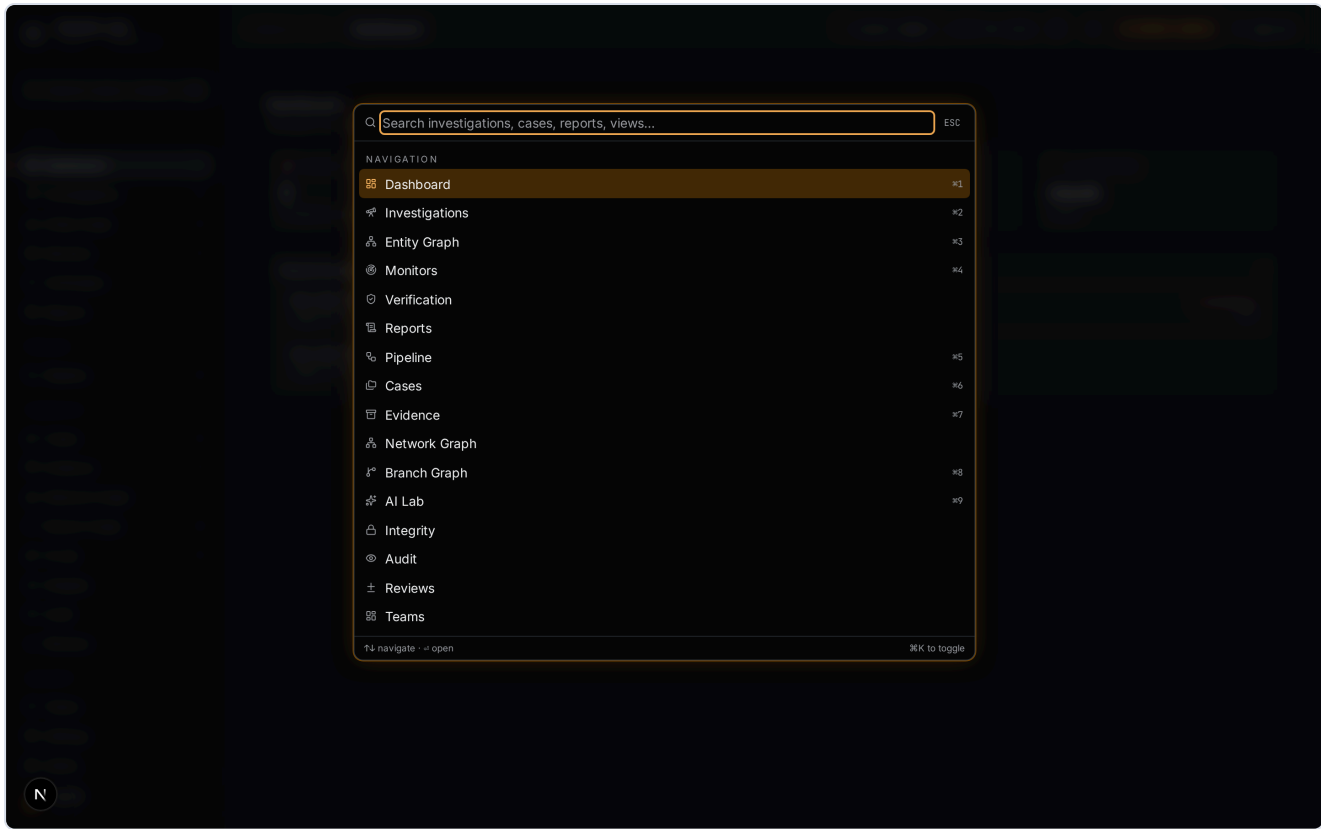
How to use the command palette (⌘K)

The single fastest way to move around once your dataset gets non-trivial.

Steps.

1. Press **⌘K** (macOS) or **Ctrl+K** (Linux/Windows) anywhere.
2. Type a few letters. The palette matches against:
 - nav items (`audit` , `entity` , `monitors`),
 - investigation titles + targets,
 - case numbers + titles,
 - report titles + sources.
3. **^ / v** to navigate.
4. **Enter** to open.
5. **Esc** (or click outside) to close.

Activating an investigation or case row sets the active id *and* switches view, so the detail panel opens instantly.



How to switch views with keyboard shortcuts

Key	View
⌘1 / Ctrl+1	Dashboard
⌘2 / Ctrl+2	Investigations
⌘3 / Ctrl+3	Entity Graph
⌘4 / Ctrl+4	Monitors
⌘5 / Ctrl+5	Pipeline
⌘6 / Ctrl+6	Cases
⌘7 / Ctrl+7	Evidence
⌘8 / Ctrl+8	Branch Graph
⌘9 / Ctrl+9	AI Lab

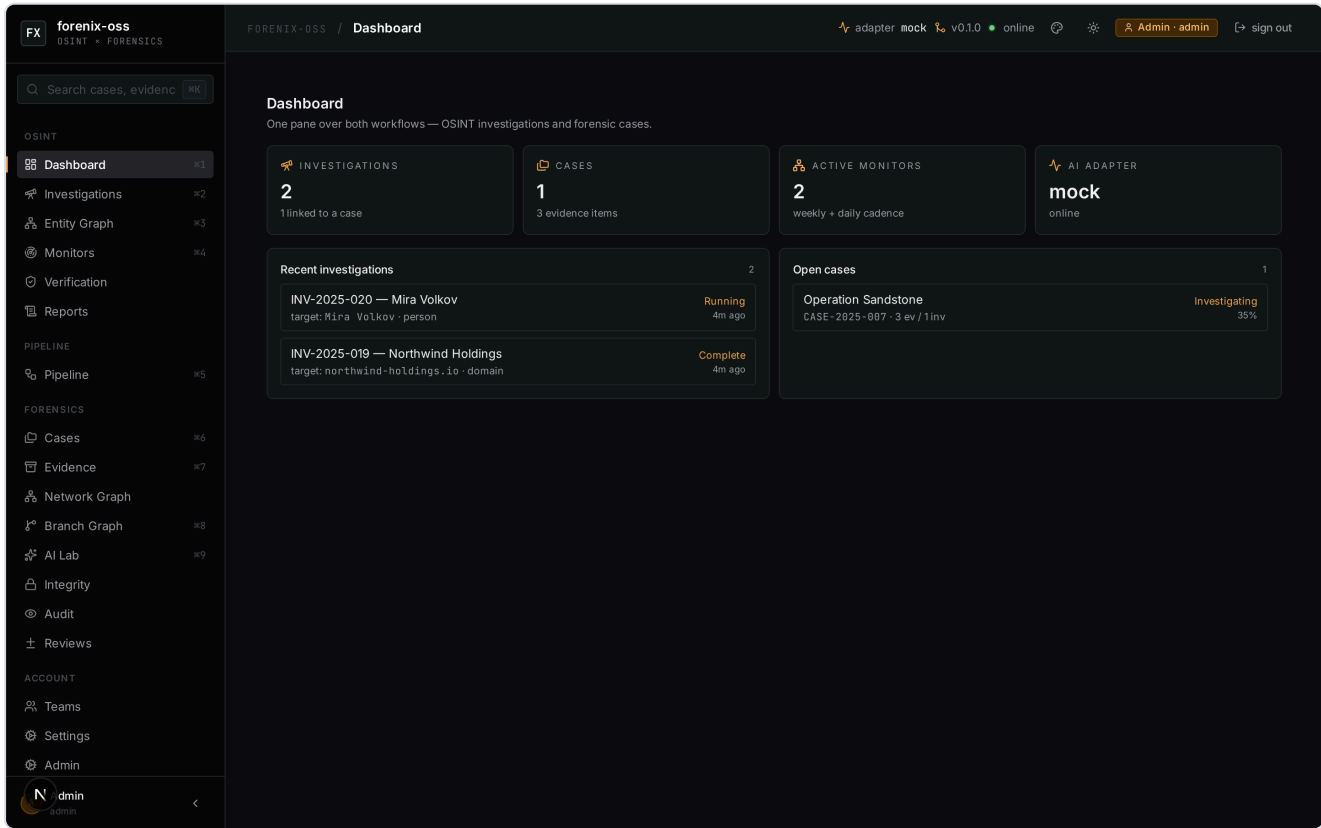
The remaining views (Verification, Reports, Network Graph, Integrity, Audit, Reviews) are reachable via ⌘K or the sidebar.

How to collapse the sidebar

To free up horizontal space:

1. Click the **Collapse** button at the bottom-left of the sidebar.

The sidebar slims to just icons + the brand mark; tooltips on hover tell you which view is which.



Click the same button (now showing a right-arrow) to expand again. The collapsed state persists across reloads (Zustand `persist`).

How to deep-link to a view or detail panel

The platform reads URL query params on first load so you can paste links to specific state.

URL	Where it lands
<code>/?view=cases</code>	Cases list
<code>/?view=cases&case=<id></code>	Specific case detail
<code>/?view=investigations&inv=<id></code>	Specific investigation detail
<code>/?view=branch-graph&case=<id></code>	Branch graph for one case
<code>/?view=dashboard&palette=1</code>	Dashboard with command palette already open

Use this when sharing screenshots, in bug reports, or in a Slack message that says "look at this case."

Setup + ops

How to switch the active AI adapter

Permanent - for every request:

1. Open `.env`.
2. Change the `AI_ADAPTER` line:

```
AI_ADAPTER=nvidia      # was 'mock'
```

3. If you're switching to a hosted adapter, make sure its key is present:

```
NVIDIA_API_KEY=nvapi-...  
NVIDIA_MODEL=meta/llama-3.3-70b-instruct
```

4. Restart the dev server: **Ctrl+C** in the terminal where `bun run dev` is running, then `bun run dev` again.

Per-request - for one pipeline run:

Add an `adapter` field to the POST body:

```
curl -X POST -H "content-type: application/json" \  
  -d '{"agentGroups":["identity"],"adapter":"openrouter"}' \  
  http://localhost:3000/api/pipeline/run/$INV
```

This overrides `AI_ADAPTER` for that single call. The top-bar chip still shows the env-default adapter; only that one request runs through the override.

How to re-seed the database

To reset to a known clean demo state:

```
bun run db:seed
```

This wipes every row via Prisma `deleteMany` and re-seeds:

- 3 users,
- 2 investigations (one bridged to a case),
- 1 case with 3 evidence items + 2 branches + 6 commits,
- 6 findings (2 already linked to evidence),
- monitors, verifications, reports, agents, tasks,
- 9 audit rows with a valid SHA-256 chain.

The seed script is safe to run any time - it doesn't drop the database file, just clears it.

Do not run `bun run db:reset` (which uses `--force-reset`) from inside an AI session unless you explicitly want to wipe the schema as well; the Prisma CLI will refuse without explicit consent.

How to recover from a "pipeline run hung" state

If a pipeline POST hangs longer than 90 s, the platform aborts the adapter call. Symptoms:

- Investigation stuck in `status: "running"`.
- Toast says *"Pipeline complete"* never appears.

Recovery.

1. Reload the Investigations view. If the row's status is `running`, the pipeline aborted server-side.
2. Pick a faster model in `.env`:

```
OPENROUTER_MODEL=openai/gpt-oss-120b:free
```

3. Restart the dev server.
4. Re-run with fewer agent groups (`identity` alone first).
5. If you need to manually reset the status, drop into a Prisma client and run:

```
await prisma.investigation.update({
  where: { id },
  data: { status: "draft" },
});
```

This write also gets audited.

This is a living document. If a workflow you need isn't here, file an issue at github.com/thunderstornX/forenix-oss/issues so the next release ships with a recipe for it.